

MBTInduce: MBTInduce Software Design

Seungjoo Ham - seungjoo.ham@stonybrook.edu

Jibeom Ryu - jibeom.ryu@stonybrook.edu

1. Introduction

1.1. Product Description

MBTInduce is an AI-powered conversational agent platform designed to bridge the gap between personality theory and real-world interaction. While most MBTI services simply provide static descriptions of personality types, our service allows users to receive direct advice, planning help, or emotional support through the specific lens and voice of a chosen MBTI type.

In daily life, people seek guidance for dilemmas, relationships, or career paths. However, everyone's needs are different. Some look for warm empathy, while others prioritize cold, logical solutions. Standard AI chatbots often lack this personal nuance, and existing MBTI apps usually stop at giving users a personality report. MBTInduce was created from a simple question: 'How would a person with a specific personality react to the current situation?'

By selecting a specific MBTI type, users can see how perspectives shift. For example, an ENFJ might offer encouragement and emotional validation regarding a career struggle, whereas an INTJ would focus on long-term efficiency and strategic planning. This allows users to experience different cognitive styles firsthand. Beyond just being fun, our goal is to provide a more tailored, satisfying interaction that feels personally resonant.

Key features include those three. First, MBTI-Driven AI responses. It generates tailored advice based on 16 personality frameworks. Second, scenario simulation. It lets users test how different types react to specific situations like dating, workplace conflicts, or team projects. Last, multi-type comparison. It allows users to compare responses from different types side-by-side to see the contrast in perspectives.

1.2. Scope

MBTInduce is developed as a responsive web-based service, ensuring accessibility across desktops, tablets, and mobile devices without the need for an app installation. The scope focuses on generating dynamic AI responses, providing empathetic feedback, and facilitating multi-type comparisons.

MBTInduce is primarily designed for individuals looking for personalized emotional support or practical advice that truly resonates with their own personality traits. Our team also sees this as a highly practical tool for students, educators, and communicators who want to improve their interpersonal skills by simulating and practicing conversations with a wide variety of personality types.

Our team designed MBTInduce to be highly versatile across many different conversational contexts. Because it is built on a flexible AI framework, the system can adapt to almost any scenario ranging from casual daily worries to complex professional dilemmas or relationship issues. This ensures that our core value of personality-driven empathy remains consistent and effective regardless of the specific topic being discussed.

What MBTInduce does: It processes user inputs regarding concerns, questions, or scenarios. It generates distinct responses based on the traits of 16 MBTI types. It enables comparative views and extended dialogues with specific personalities.

What MBTInduce does not do: Our team is not a professional psychological counseling or mental health treatment service team. So MBTInduce does not offer clinical personality diagnoses or scientifically validate MBTI theory. The service is intended for self-exploration, communication practice, and entertainment, not as a replacement for professional therapy.

1.3. Users

We have four target users. First, empathy seekers. We seek users who have a clear sense of their own personality and want to receive support in a way that feels natural and validating to them. Second, general MBTI enthusiasts. We seek people who enjoy personality tests as a hobby and want to explore how different types behave in what-if scenarios. Third, students and communicators. We seek those looking to improve their interpersonal skills by understanding how people with different traits might perceive the same situation. Last, content creators. We seek writers or role-players who need a reference tool to build consistent character voices and dialogue based on personality archetypes.

1.4. User Feedback Plan

To ensure we are building something people actually want to use, we plan to gather feedback early and often. We will present our team's project proposal, UI mockups, and core requirements to potential users and conduct structured interviews. Instead of just asking if they like it, our team wants to find out in what specific situations they would open this service whether it's after a fight with a friend or when they're stuck on a career decision.

1.5. Existing Alternatives

Our team analyzed four existing platforms to define the unique value proposition. First, 16Personalities. It is great for static information and testing but they lack interaction. Second, Boo. Boo is innovative in connecting people via personality but its focus is social matching rather than deep, scenario-based AI consultation. Third, Character.AI. It offers great interaction, but it isn't structured around the MBTI framework, making it hard to compare specific personality traits systematically. Last, Generic AI ChatGPT. While powerful, it requires the user to write complex prompts to get a specific personality voice.

Our team noticed that popular sites like 16Personalities are most limited to providing static reports. Our team decided to move beyond just reading about a type and instead focused on interactive, real-time AI responses so users can actually experience a personality in action. Our team found that apps like Boo or Character.AI make it difficult to see the differences between types side-by-side. To fix this, our team incorporated a multi-type comparison view. This allows users to see exactly how different cognitive styles contrast within the exact same situation. While ChatGPT is a power tool, its blank slate nature can be a barrier for the average user who doesn't know how to write complex prompts. Our team solved this by building a pre-structured interface where users can trigger specific MBTI behaviors with a simple click, making the technology much more accessible.

1.6. Definitions

AI generated response is a text created by the AI model based on user input. Empathy-based response is an answer focused on emotional validation rather than just raw information. Scenario simulation is seeing how a specific type reacts to a predefined or user-defined context such as how an ESFP breaks up. Multi-type comparison is displaying responses from two or more types simultaneously for easy contrast. Responsive web is a design that adjusts seamlessly to any screen size.

1.7. References

16 Personalities. (n.d.). Our Theory: NERIS Type Explorer. Retrieved from <https://www.16personalities.com/articles/our-theory>

Boo. (n.d.). The Psychology of Personality in Social Media. Retrieved from <https://boo.world/about>

Character.AI. (n.d.). How Character AI works: Language Models and Personas. Retrieved from <https://character.ai/help>

OpenAI. (2023). GPT-4 System Card: Tailoring model behavior through system prompts. OpenAI. <https://openai.com/research/gpt-4>

2. Requirements

2.1 Functional Requirements

- Main Chat Interaction
 - Primary Actor: End User
 - The system shall allow users to input text prompts and receive AI-generated responses in a chat interface.
 - The system shall display conversation history including both user messages and AI responses.
 - The system shall support an empty state when no conversation exists.
- MBTI-based Response Configuration
 - Primary Actor: End User
 - The system shall allow users to configure MBTI personality traits using adjustable sliders (E/I, S/N, F/T, P/J).
 - The system shall display percentage-based intensity for each trait.
 - The system shall generate AI responses reflecting selected MBTI configurations.
 - The system shall allow users to generate multiple responses for comparison ("Show Both" feature).
- Multi-Perspective AI Comparison
 - Primary Actor: End User
 - The system shall allow users to generate and compare multiple AI responses based on different MBTI configurations.
 - The system shall display responses side-by-side for comparison.
- Scenario-based Conversation Simulation

- Primary Actor: End User
 - The system shall allow users to create scenarios and simulate conversations between different MBTI personalities.
 - The system shall allow users to input, edit, and manage scenarios.
 - The system shall display scenario content clearly.
- Calendar & Event Management
 - Primary Actor: End User
 - The system shall provide a calendar interface for viewing scheduled events.
 - The system shall allow users to create, edit, and delete events.
 - The system shall support multiple calendar views (Month, Week, Day, Agenda).
 - The system shall allow integration with MBTI-based planning features.
- History Management
 - Primary Actor: End User
 - The system shall store and display previous AI-generated conversation.
 - The system shall allow users to edit or delete saved records.
 - The system shall display metadata such as date and MBTI attributes.
- User Account & Profile Management
 - Primary Actor: End User
 - The system shall allow users to register and log in.
 - The system shall allow users to manage profile information (nickname, MBTI).
 - The system shall persist user preferences for personalized interaction.
- Web Integrated Interface
 - Primary Actor: End User
 - The system shall provide a multi-panel web interface integrating chat, MBTI controls, and navigation.
 - The system shall allow real-time interaction without screen transitions.
 - The system shall allow users to adjust MBTI settings dynamically during chat.
- Prompt Template System
 - Primary Actor: End User
 - The system shall provide predefined prompt templates categorized by intent.
 - The system shall allow users to insert templates into input fields.
 - The system shall generate AI responses based on selected templates.
- Admin Panel
 - Primary Actor: Admin
 - The system shall allow administrations to monitor user statistics and feedback.
 - The system shall allow management of prompt templates (create, edit, delete).
 - The system shall display analytics such as count and response ratings.
- Stretch Goals
 - AI-based Schedule Recommendation

- The system shall generate schedule suggestions based on MBTI traits.
 - The system shall recommend optimal task planning styles.
- Multi-user Scenario Simulation
 - The system shall simulate conversations among multiple MBTI personalities simultaneously.
 - The system shall allow users to assign multiple roles within a scenario.

2.2 Use Cases

- UC-1: Generate AI response with MBTI configuration
 - Primary Actor: End User
 - Priority: Essential
 - Scenario:
 1. The user navigates to the Main Chat screen.
 2. The user inputs a text prompt.
 3. The user configures MBTI traits. (optional)
 4. The user presses "Generate".
 5. The system sends requests to the AI backend.
 6. The system receives an AI response.
 7. The system displays responses in chat.
 - Extensions:
 - 3a. User skips MBTI configuration -> default personality used.
 - 5a. AI response fails -> system displays error message.
- UC-2: Compare Multiple MBTI Responses
 - Primary Actor: End User
 - Priority: Essential
 - Scenario:
 1. The user navigates to the MBTI configuration screen.
 2. The user selects both MBTI traits for comparison.
 3. User clicks "Generate".
 4. The system generates multiple responses.
 5. System displays responses side-by-side
 - Extensions:
 - 3a. Invalid configuration -> system prompts correction.
- UC-3: Create and Manage Scenario Simulation
 - Primary Actor: End User
 - Priority: Expected
 - Scenario:
 1. The user navigates to the Scenario screen.
 2. User clicks "Make New".
 3. User inputs scenario description.
 4. Users assign MBTI personalities.
 5. The system generates simulated conversation.
 6. The system displays results.
- UC-4: Manage Calendar Events

- Primary Actor: End User
- Priority: Expected
 - Scenario:
 1. The user opens the Calendar screen.
 2. The user selects a date.
 3. User inputs event details.
 4. User clicks “Submit”.
 5. The system saves and displays events.
 - Extensions:
 - 4a. User deletes event -> system removes event.
- UC-5: Access and Manage History
 - Primary Actor: End User
 - Priority: Essential
 - Scenario:
 1. The user navigates to the History screen.
 2. The system displays saved conversations.
 3. The user selects a record.
 4. Users edit or delete records.
 5. System updates history.

2.3 User Interfaces

Figma link :

<https://www.figma.com/design/33GLdlpEeiUtmVXV6Ff0oA/MBTInduce?node-id=0-1&t=GGWTgn5DrGEXUuTR-1>

The system provides both mobile (iOS) and web-based interfaces to support user interaction with AI-driven MBTI-based responses, scenario simulation, and planning features. The user interface is designed to be intuitive, responsive, and consistent across different platforms.

The Main Chat Screen serves as the primary interface for user interaction with the AI system.

It includes:

- A text input field where users can enter prompts or messages
- A conversation display area showing both user inputs and AI-generated responses
- A bottom navigation bar for accessing other features
- A menu button for additional navigation options

On the Main chat Screen, the screen allows users to customize the personality traits that influence AI responses.

It includes:

- Four slider controls representing MBTI dimensions (E/I, S/N, F/T, P/J)
- Percentage indicators showing the intensity of each selected trait
- A “Generate” button to create AI responses based on selected traits

- A comparison mode (“Show Both”) that enables users to generate and compare multiple responses

The interface is designed to provide real-time feedback and intuitive control over personality configurations.

The Simulation Screen enables users to simulate conversations between different MBTI personalities within a custom context.

It includes:

- A scenario input area where users can describe a situation
- A “Make New” button to create a new scenario
- A display area showing the current scenario and generated conversations
- Editing functionality to modify scenario content

This screen allows users to explore how different personality types interact in various situations.

The Calendar Screen provides scheduling and planning functionalities.

It includes:

- A calendar interface displaying events in multiple views (Month, Week, Day, Agenda)
- Navigation controls such as Today, Back, and Next
- A visual display of scheduled events

The Event Management interface includes:

- Input fields for date and time selection
- An option for all-day events
- A description field for event details
- Buttons for saving and deleting events

This feature can be extended to support AI-assisted planning based on MBTI preferences.

The History Screen allows users to manage previously generated AI interactions.

It includes:

- A list view of saved conversations
- A preview of each record
- Metadata such as date and MBTI attributes
- Options to edit or delete saved records

This interface supports efficient organization and retrieval of past interactions.

This interface provides predefined prompt templates to guide user interaction.

It includes:

- A list of categorized templates (e.g., emotional, logical, analytical)
- The ability to select and insert templates into the input field
- Direct submission of prompts based on selected templates

This feature improves usability and helps users generate more effective AI interactions.

The Admin Panel supports system monitoring and content management.

It includes:

- A dashboard displaying system metrics (user count, query count, average ratings)
- A feedback analysis section
- A prompt template management system (add, edit, delete)

The admin interface is available on both mobile and web platforms and is designed for efficient system oversight.

2.4 Non-functional Requirements

- Performance
 - The system shall generate AI responses within 3 secs on average under normal conditions.
 - The system shall support at least 100 concurrent users without degradation.
- Usability
 - The system shall provide an intuitive UI with clear navigation (chat, calendar, history).
 - The system shall support both mobile and web platforms.
- Reliability
 - The system shall ensure 99% uptime excluding maintenance.
 - The system shall recover gracefully from API failures.
- Security
 - The system shall securely store user data using encryption.
 - The system shall implement authentication (OAuth / session-based).
 - the system shall prevent unauthorized access to user data.
- Scalability
 - The backend shall support horizontal scaling.
 - The system shall handle increasing user load without major redesign.
- Maintainability
 - The system shall follow modular architecture (frontend/backend separation).
 - The system shall support easy updates to AI logic and UI components.
- Compatibility
 - The system shall support modern browsers: Chrome, Safari.
 - The system shall support iOS mobile devices.

3. System Architecture

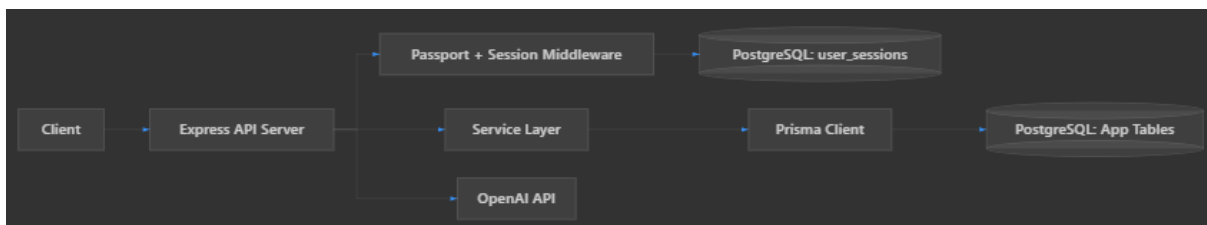
3.1 Overview

- Frontend
 - The frontend of MBTInduce will be implemented using React 19.2.0, TypeScript 5.9.3, and Vite 7.3.1. The frontend is responsible for providing the main user interface of the system, including the chat screen, MBTI control panel, simulation screen, navigation structure, and calendar-based scheduling feature. Since MBTInduce depends heavily on user interaction, such as adjusting MBTI values, selecting response styles, sending chat messages, rating AI responses, and viewing scheduled activities, the frontend must be responsive, maintainable, and easy to expand.
 - The project uses React as the main library because it allows the interface to be divided into reusable components. This is important for the system because many parts of the interface, such as buttons, sliders, chat bubbles, navigation menus, and layout sections, are repeated across different screens. By separating the UI into components, the code becomes easier to manage and modify as the project grows. The current project structure also follows a component-based organization, with UI elements separated into folders such as atoms, molecules, organisms, and templates. This structure is similar to the atomic design pattern, where small UI elements are combined to create larger and more complex interface sections. This design approach helps keep the frontend organized and supports future expansion.
 - TypeScript is used with React to improve code reliability. Since the frontend needs to handle different types of data, such as MBTI slider values, selected calendar dates, chat messages, user inputs, and AI response ratings, TypeScript helps define clear data structures and reduces mistakes during development. It also makes collaboration easier because developers can understand what kind of data each component expects.
 - For the development environment and build process, the project uses Vite. Vite was chosen because it provides a fast development server, quick hot module replacement, and a simple build configuration. Compared to older React build tools, Vite is lighter and faster, which makes it suitable for a student project that needs rapid UI development and frequent testing.
 - For styling, the frontend uses Emotion 11.14. Emotion allows us to write component based styles directly in the React and TypeScript environment. This is useful because the MBTInduce interface has several custom designed sections, including the main chat area, side control panel, simulation page, and calendar layout. Emotion also makes it easier to use shared theme values, such as colors, spacing and layout rules, across the whole application. This helps keep the visual design consistent.
 - The project also uses React Router DOM 7.14.1 for client side routing. React Router will manage page navigation between different parts of the application, such as the main chat page, simulation page, and calendar page. This allows the application to behave like a single-page application, where users can move between screens without fully reloading the website.
 - In addition, our team is adding React Big Calendar for the calendar feature. This library will be used to display schedules and selected dates in a calendar

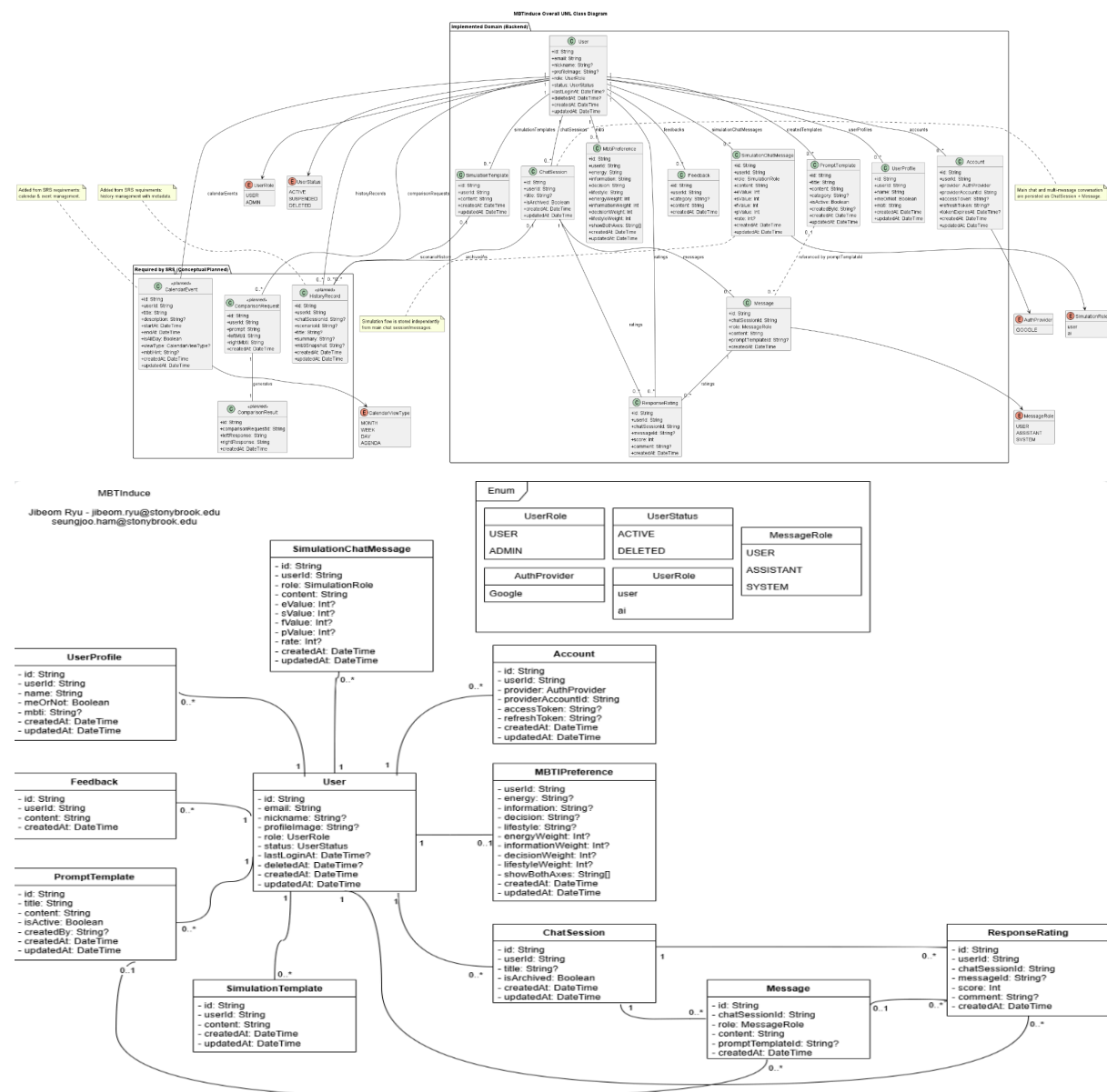
format. Instead of building a full calendar system from scratch, React Big Calendar provides pre-built calendar views and event display features. In MBTInduce, this will be useful for showing user-selected dates, planned activities, or simulation-related schedules in a clear visual format. This improves the usability of the system because users can understand date-based information more easily through a calendar interface.

- Overall, this technology stack was chosen because it supports fast development, strong maintainability, and interactive UI design. React makes it easier to build reusable and dynamic interface components and TypeScript improves the stability of the code by catching type-related errors early. Vite improves the development experience by making the project faster to run and build. Emotion gives more flexibility for custom styling, which is important because MBTInduce has a unique interface based on MBTI controls and chat interaction. React Router DOM supports smooth page navigation, and React Big Calendar helps to add a complex calendar feature efficiently without spending unnecessary time building every calendar function manually.
- Backend + Database
 - Major components of the system
 - API Server: [Node.js](#) + REST API based on Express
 - Auth/Session layer: Passport(Google OAuth 2.0) + express-session
 - Session storage: Use the PostgreSQL user_sessions table via connect-pg-simple
 - Data Access Layer : Prisma ORM
 - PostgreSQL
 - Implementation Languages/Technologies and Key Versions
 - Lang/Runtime
 - JavaScript (ES Modules)
 - [Node.js](#)(Runtime versions depend on deployment and development environment configurations)
 - Web Framework / Middleware
 - express 4.21.2
 - cors 2.8.5
 - helmet 8.0.0
 - morgan 1.10.0
 - dotenv 16.4.7
 - Authentication / Session Management
 - passport 0.7.0
 - passport-google-oauth20 2.0.0
 - express-session 1.19.0
 - connect-pg-simple 10.0.0
 - DB/ORM
 - prisma 6.16.2
 - @prisma/client 6.16.2
 - DB Provider: PostgreSQL (schema.prisma)
 - External API Integration
 - openai 6.34.0
 - Key Third-Party Components and Their Purposes

- Express: The core web framework for API routing, middleware chaining, and error handling.
- Prisma: Type-safe database access, model-based CRUD operations, and migrations (prisma migrate).
- PostgreSQL: Persistent data storage for users, chats, simulations, and feedback.
- Passport Google OAuth 2.0: Handles the social login authentication flow.
- express-session + connect-pg-simple: Reliable session management using server/DB-based storage.
- Helmet/CORS/Morgan: Enhances security headers, CORS policies, and request logging for better operational reliability.
- OpenAI SDK: Invokes LLMs to generate MBTI-driven responses.
- Applied Design Patterns / Architectural Style
 - The backend of this project follows a layered architecture similar to MVC.
 - routes → endpoint mapping
 - controllers → request validation and response formatting
 - services → business logic and Prisma calls
 - lib/config/middlewares → shared infrastructure
 - In summary, this structure follows a typical Route-Controller-Service-DB pattern, separating responsibilities to improve maintainability.
- Rationale for Technology Stack Selection
 - Node.js + Express: Enables rapid API development and iteration, with a mature middleware ecosystem.
 - Prisma + PostgreSQL: Supports schema-driven development and ensures relational data integrity.
 - Passport + Session-based authentication: Provides a simple and reliable way to handle OAuth login and user state management.
 - PostgreSQL session storage: Maintains session consistency across server restarts and multi-instance environments .
 - Security/operational middleware (Helmet/CORS/Morgan): Meets essential requirements for production systems.

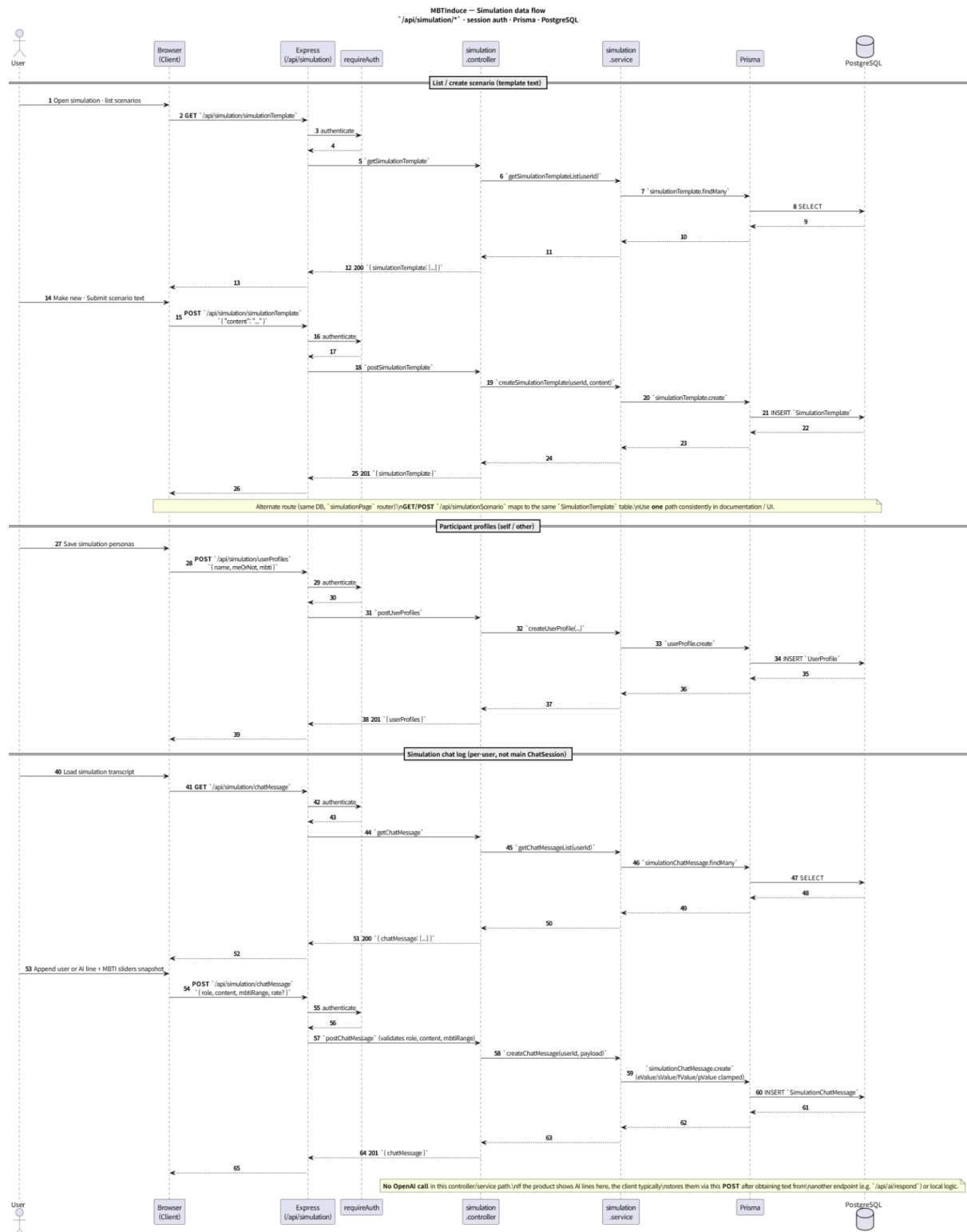


3.2 Data Design



3.3 UML Sequence Diagram

Simulation flow



```

sequenceDiagram
    participant User
    participant Browser as Browser (Client)
    participant Express as Express (/api/chat)
    participant requireAuth
    participant chatCompat as chat.compat.controller
    participant postMessageCore as postMessageCore (chat.controller)
    participant openAIService as openAIService (getChatCompletion)
    participant Prisma
    participant PostgreSQL
    participant OpenAIAPI as OpenAI API

    Note over User, Browser: 1 Open main chat UI
    Browser->>Express: 2 GET "/api/chat" Cookie session
    Express->>requireAuth: 3 authenticate
    requireAuth-->>Express: 4
    Express->>chatCompat: 5 "getMainChatFlat"
    Note over chatCompat: Load main chat (flat array)
    chatCompat->>openAIService: 6 "findFirst" / "create" ChatSession (title "Main Chat")
    openAIService->>Prisma: 7 SELECT / INSERT "ChatSession"
    Prisma-->>openAIService: 8
    openAIService-->>chatCompat: 9
    chatCompat->>openAIService: 10 "message.findMany" + "mbtiPreference" + "responseRating" (latest scores)
    openAIService->>Prisma: 11 SELECT
    Prisma-->>openAIService: 12
    openAIService-->>chatCompat: 13
    chatCompat->>Express: 14 200 JSON "ChatMessage[]"
    Express-->>Browser: 15 Render messages

    Note over User, Browser: 16 Type message - Send
    Browser->>Express: 17 POST "/api/chat" JSON "content", optional "mbtiRange" / weights ... Cookie session
    Express->>requireAuth: 18 authenticate
    requireAuth-->>Express: 19
    Express->>chatCompat: 20 "postMainChatFlat"
    Note over chatCompat: Send message (user + AI reply persisted)
    chatCompat->>openAIService: 21 "getOrCreateMainChatSession" (same as above)
    openAIService->>Prisma: 22
    Prisma-->>openAIService: 23
    openAIService-->>chatCompat: 24
    chatCompat->>postMessageCore: 25 "postMessageCore(userId, sessionId, body)"
    postMessageCore->>openAIService: 26 Load / upsert "MbtiPreference" (optional)
    openAIService->>Prisma: 27
    Prisma-->>openAIService: 28
    openAIService-->>chatCompat: 29
    chatCompat->>postMessageCore: 30 "message.create" role USER
    postMessageCore->>Prisma: 31 INSERT "Message"
    Prisma-->>postMessageCore: 32
    postMessageCore-->>chatCompat: 33
    chatCompat->>postMessageCore: 34 "message.findMany" (history, capped)
    postMessageCore->>Prisma: 35 SELECT
    Prisma-->>postMessageCore: 36
    postMessageCore-->>chatCompat: 37
    chatCompat->>openAIService: 38 "getChatCompletion(messages)"
    openAIService->>Prisma: 39 POST ".../v1/chat/completions" (axios, Bearer key)
    Prisma-->>openAIService: 40 assistant text
    openAIService-->>chatCompat: 41
    chatCompat->>postMessageCore: 42 "message.create" role ASSISTANT
    postMessageCore->>Prisma: 43 INSERT "Message"
    Prisma-->>postMessageCore: 44
    postMessageCore-->>chatCompat: 45
    chatCompat->>postMessageCore: 46 "chatSession.update" ("updatedAt")
    postMessageCore->>Prisma: 47 UPDATE "ChatSession"
    Prisma-->>postMessageCore: 48
    postMessageCore-->>chatCompat: 49
    chatCompat->>postMessageCore: 50 "{ userMessage, assistantMessage, appliedMbti }"
    postMessageCore->>Prisma: 51 Rebuild flat list ("buildFrontendChatArray")
    Prisma-->>postMessageCore: 52 SELECT messages + ratings
    postMessageCore-->>chatCompat: 53
    chatCompat->>Express: 54 200 JSON "ChatMessage[]"
    Express-->>Browser: 55 Updated thread

    Note over User, Browser: 57 Rate a reply (1-5)
    Browser->>Express: 58 PATCH "/api/chat/messageId" JSON "{ 'rate': 1-5 }"
    Express->>requireAuth: 59 authenticate
    requireAuth-->>Express: 60
    Express->>chatCompat: 61 "patchMainChatMessageRate"
    Note over chatCompat: Rate assistant message (optional)
    chatCompat->>openAIService: 62 validate message - upsert "ResponseRating"
    openAIService->>Prisma: 63 SELECT / INSERT or UPDATE
    Prisma-->>openAIService: 64
    openAIService-->>chatCompat: 65
    chatCompat->>openAIService: 66 "buildFrontendChatArray" again
    openAIService->>Prisma: 67
    Prisma-->>openAIService: 68
    openAIService-->>chatCompat: 69
    chatCompat->>Express: 70 200 JSON "ChatMessage[]"
    Express-->>Browser: 71
  
```

MBTInduce - Main chat flow

GET/POST/PATCH /api/chat - session auth - PostgreSQL + OpenAI

MBT-weighted system prompt is built in "postMessageCore" ("mbtiToWeightedInstruct", recent history window).

3.4 API Design

https://docs.google.com/spreadsheets/d/1kF9JzOzi3R0BXaf_N1rMrUaFRsz3cgdRVHgJUSEb_VY/edit?usp=sharing

3.5 Deployment

The MBTInduce application will be deployed using Railway as the primary cloud platform. Both the frontend and backend will be deployed as separate services on Railway, allowing each component to be managed, updated, and scaled independently. This separation improves system stability and flexibility, especially as the project grows. For the frontend deployment, a dedicated Railway service will be created. The frontend is built using React, TypeScript, and Vite, and the production build will generate optimized static files (HTML, CSS, and JavaScript). These files will be served through the Railway after the build process. Railway is integrated with GitHub, so whenever code is pushed to the repository, it automatically triggers a new build and deployment. This enables continuous deployment without requiring manual steps and allows rapid iteration during development. Railway also supports easy environment variable configuration, logging, and automatic deployment, which simplifies server management. By deploying the backend independently from the frontend, our team can update or scale the server without affecting the client-side application. The deployment process follows a CI/CD (Continuous Integration and Continuous Deployment) workflow. When our team pushes code to the GitHub repository, Railway automatically builds and deploys the frontend and backend service. Then the updated application becomes immediately available online. This automated workflow reduces deployment errors and improves development efficiency. Railway was chosen because it provides a simple and unified deployment environment for both frontend and backend services. It eliminates the need for complex server configuration while still offering essential features such as automatic deployment, environment variable management, and logging. This makes it especially suitable for a project like MBTInduce, where fast development and easy deployment are important.

3.6 Code Conventions

- Google JavaScript Style Guide
 - <https://google.github.io/styleguide/jsguide.html>
- Prisma formal style(model PascalCase, field camelCase)
 - <https://www.prisma.io/docs/orm/prisma-schema/overview>
- For the frontend, React components use PascalCase, such as MainChatPage, CalendarScreen, RightPanel, or ChatMessage. Variables, functions, event handlers, and state values will use camelCase, such as selectedDate, handleSubmit, setUserInput, or fetchChatMessages. TypeScript interfaces and types will also use PascalCase, such as ChatMessageType, MbtiRange, or CalendarEvent.
- For the backend, files and folders use clear lowercase or camelCase names depending on the purpose. API route handlers, controller functions, service functions and variables will use camelCase, such as createChatMessage, getUserProfile, generateMbtiResponse, or saveFeedback. Database models managed through

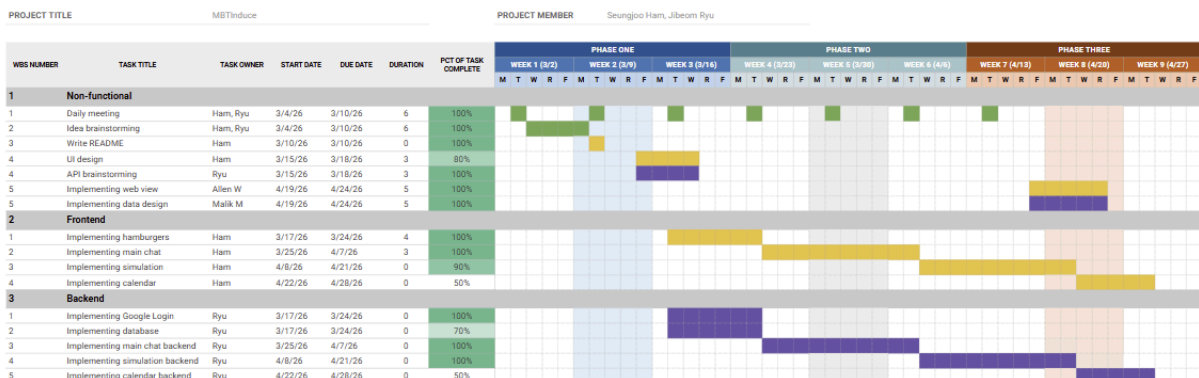
Prisma will use PascalCase, such as User, ChatMessage, Feedback or CalendarEvent.

- Constants and environment-related values use UPPER_SNAKE_CASE, such as DATABASE_URL, OPENAI_API_KEY, GOOGLE_CLIENT_ID, and SESSION_SECRET.
- We use consistent formatting across the project
 - Use 2 spaces for indentation
 - Use semicolons at the end of statements
 - Prefer single quotes for strings

4. Schedule

So far

GANTT CHART



In the future

- Project milestone 1 **5/11/2026**
 - Hamburgers, Main chat, Simulation, Calendar, History, Sign up, my page by Ham
 - Google Login Implementation, Database (including creating variable names to send and receive) - (his) mainchat, simulation, calendar, Main Chat, Simulation, Calendar, History, sign up page, my page by Ryu
- Project milestone 2 **5/13/2026**
 - Admin (Total users, average rating, total questions window, rating window, question template window, log out button) by Ham
 - Administrators page by Ryu
- Project milestone 3 **5/20/2026**
 - Reaction type by Ham
 - BACK ENDLAYER - MAIN CHAT, SIMULATION, CALENDAR by Ryu
 - ai Service Layer - Tuning by Ryu
- Project milestone 4 (beta release) **5/27/2026**
 - Distribution by Ham

5. Contributions

Jibeom Ryu - Jibeom wrote 1 & 2, 3.1, 3.2, 3.3, 3.4, 4

Seungjoo Ham - Seungjoo wrote 3.1, 3.5, 3.6, 4